

# L'agent contabile. Costruirlo live.

La guida pratica passo-passo per costruire, davanti alla camera, l'agent che fa la contabilità del tuo store: un solo agent, due modalità — dashboard live nel browser e report giornaliero su Drive e Telegram.

- 01 Cosa costruiamo — un agent, due modalità
- 02 Prerequisiti — le credenziali da avere prima
- 03 config.json — il cuore di tutto
- 04 Build con la agent factory — la spec dettata
- 05 Modalità report — il flusso del run
- 06 Modalità live — la dashboard
- 07 Schedulazione + deploy online
- 08 Le regole dure
- 09 Checklist del build live

# Un agent, due modalità

Non costruiamo due programmi: costruiamo **un solo agent contabile** che lavora in due modalità, dalle stesse fonti dati e con lo stesso `config.json`.

|                                                                                                                                                                                              |                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MODALITÀ · LIVE                                                                                                                                                                              | MODALITÀ · REPORT                                                                                                                                              |
| <b>Dashboard</b><br>Web server sempre acceso. Un poller rilegge i dati ogni 120s e li tiene in cache. Espone <code>/api/metrics</code> e una pagina con login: i KPI in diretta nel browser. | <b>Giornaliero</b><br>Schedulato 1 volta al giorno (16:00). Genera l'Excel a 6 fogli, lo salva per anno/mese, lo carica su Drive e manda il recap su Telegram. |

**Le fonti dati** (cloud, condivise dalle due modalità): **Shopify Admin API** (ordini, refund, fee, clienti), **Triple Whale** (spesa ads, ROAS), **Google Drive** (XLSX costi fornitore), **Telegram** (notifiche), e — opzionale — **Supabase** (storico KPI).

# Le credenziali da avere prima

Procurati questi prima di iniziare il build. Senza le prime quattro, niente funziona.

| COSA                         | DOVE SI PRENDE                                                                                                                       | VARIABILE / FILE                                                              |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Shopify Admin API token      | Shopify → Settings → Apps → Develop apps → token <code>shpat_...</code>                                                              | <code>SHOPIFY_API_PASSWORD</code>                                             |
| Shopify shop url             | es. <code>tuostore.myshopify.com</code>                                                                                              | <code>SHOPIFY_SHOP_URL</code>                                                 |
| Triple Whale API key         | Triple Whale → Settings → API                                                                                                        | <code>TRIPLE_WHALE_API_KEY</code>                                             |
| Google Service Account JSON  | Google Cloud Console → IAM → Service Account → chiave JSON. Poi <b>condividi le cartelle Drive con l'email del service account</b> . | <code>google_service_account.json</code>                                      |
| Google Drive folder IDs      | gli ID delle cartelle (root, "Giornalieri", "CSV costi")                                                                             | <code>GDRIVE_ROOT_FOLDER_ID</code> , <code>GDRIVE_ACCOUNTING_FOLDER_ID</code> |
| Telegram bot token + chat id | @BotFather crea il bot → token; chat id col bot                                                                                      | <code>TELEGRAM_BOT_TOKEN</code> , <code>TELEGRAM_CHAT_ID</code>               |
| Gmail app password           | Google Account → Sicurezza → password per app (per leggere il CSV costi del fornitore via email)                                     | <code>config.email_watcher</code>                                             |
| Login dashboard              | utente/password che scegli tu                                                                                                        | <code>LIVEKPI_USER</code> , <code>LIVEKPI_PASS</code>                         |

## Il cuore di tutto

Un solo `config.json` serve all'agent in entrambe le modalità. Le sezioni principali:

```
# config.json - sezioni
shopify      # shop_url, api_key, api_password, api_version
triple_whale # api_key, shop_url, currency (EUR)
google_drive # credentials_file, root_folder_id, folder IDs
cogs         # matrix[categoria][paese][quantità] = COGS per ordine
products     # {id_prodotto: nome, label, prezzo, cogs}
finance      # fee gateway, soglie ROAS
exchange_rates # cambi valuta (fallback)
email_watcher # gmail + app_password (legge il CSV costi)
report       # company_name, file_prefix, timezone
```

La **cogs.matrix** è la tabella costi per prodotto/paese/quantità: è la fonte dei COGS quando manca il CSV del fornitore. Va riempita coi tuoi costi reali, 1:1.

## Build con la agent factory — la spec

Stesso metodo del corso: la agent factory. Lancia `/agent-spec` e detta il contratto per intero.

Le hard rules vanno **dentro** la spec: l'agent nasce già con le regole.

```
# dentro Claude Code:
/agent-spec

# detti, a voce:
"Voglio un agent contabile per il mio store: un solo programma, due modalità.

MODALITÀ REPORT (1x/giorno, schedulato alle 16:00):
1) legge gli ordini Shopify del giorno (filtro processed_at, esclude €0)
2) legge i costi dal CSV del fornitore (o dalla matrix COGS in config)
3) calcola revenue, COGS, gross profit, net, BE ROAS + ROAS da Triple Whale
4) genera un Excel a 6 fogli (Analisi, Ordini, Prodotti, Paesi, Customers, Meta)
5) lo salva in locale per anno/mese, lo carica su Drive, manda il recap su Telegram

MODALITÀ LIVE (web server sempre acceso):
- espone /api/metrics con i KPI del giorno + 90 giorni
- un poller rilegge i dati ogni 120s e li tiene in cache su disco
- una pagina HTML in stile SHIN-EN con login a cookie

HARD RULES: niente stime; STOP + notifica Telegram se manca un costo SKU;
mai sovrascrivere file (test in /TEST/); timezone Europe/Rome; 1 sola istanza."
```

Poi il giro completo: `/agent-tasks` → `/agent-execute` → `/agent-verify`. La factory costruisce, tu guardi e commenti.

## Il flusso di un run

- 1 **COGS del giorno**  
Cerca i costi in ordine: CSV del fornitore → matrix in config.  
**Regola dura:**  
se manca il CSV e non c'è la matrix, si ferma.
- 2 **Ordini Shopify**  
Legge gli ordini del giorno con filtro `processed_at` (data del pagamento), esclude gli ordini a €0.
- 3 **Calcolo COGS**  
Per ogni riga d'ordine: quantità × costo (per prodotto/paese/quantità). COGS reali, non stimati.
- 4 **Triple Whale + refund**  
Spesa ads (Meta/TikTok/Google), ROAS attribuito e blended, refund del giorno, nuovi vs returning.
- 5 **Genera + distribuisce**  
Excel a 6 fogli → salva in locale (anno/mese) → carica su Drive → recap su Telegram con il link.

**Test:** lancia l'agent su una data passata in modo test — scrivi in `/TEST/` e non manda Telegram veri:  
`python main.py 2026-06-16` (o flag `LEISHAPE_TEST=1`). Verifica che crei l'Excel e lo carichi nella cartella TEST.

## La dashboard

Stesso agent, modalità web. Web server Python (stdlib, niente framework), avviato con `python server.py` sulla porta da `PORT` (default 8787). Un thread HTTP + un thread poller che ogni 120s rilegge i dati e li tiene in cache in memoria e su disco.

| ROTTA            | AUTH       | COSA FA                                   |
|------------------|------------|-------------------------------------------|
| /                | sì (login) | la dashboard HTML coi KPI                 |
| /login · /logout | pubblico   | login a cookie firmato (user/pass da env) |
| /api/metrics     | sì         | JSON con tutti i KPI (oggi + 90 giorni)   |
| /api/refresh     | sì         | forza un refresh subito                   |
| /healthz         | pubblico   | health check                              |

**Priorità delle fonti KPI:** se c'è l'XLSX costi del giorno su Drive → è la verità. Se manca (tipico per oggi/ieri) → pipeline live: Shopify + Triple Whale + matrix COGS. **Net = vendite – refund – ads – COGS – fee.**

## Schedulazione + deploy online

**Modalità report — schedulazione** (sceglie una): un **cron job del cloud** (Render Cron / Railway) che lancia `python main.py` 1x/giorno alle 16:00 — l'opzione più pulita; oppure **launchd/cron** in locale; oppure un loop `scheduler.py` come worker sempre acceso.

**Modalità live — deploy** su Render (o equivalente) come web service:

```
type: web · runtime python
buildCommand: pip install -r requirements.txt
startCommand: python server.py
healthCheckPath: /
disk: /data 1GB (persistente)
# env: SHOPIFY_*, TRIPLE_WHALE_API_KEY, GOOGLE_SERVICE_ACCOUNT_JSON (inline),
#      LIVEKPI_USER, LIVEKPI_PASS, GDRIVE_*_FOLDER_ID, DATA_DIR=/data, PORT
```

**Cold start:** al primo avvio la dashboard scarica lo storico 90 giorni (~10 min, normale). Dopo si aggiorna ogni 2 minuti. Dominio tuo via Cloudflare, accesso protetto da password.

## Le regole che fanno la differenza

### DATI

#### Niente stime

Se manca un costo SKU → STOP + notifica Telegram. Mai inventare i COGS, mai fallback a percentuali. È voluto.

### FILE

#### Mai sovrascrivere

Ogni giorno il suo file. I test vanno in `/TEST/` con prefisso `TEST_`: mai toccare i report veri.

### RUNTIME

#### TZ + 1 istanza

Timezone `Europe/Rome` (i confini di giornata dipendono da qui). La modalità live è stateful: 1 sola istanza con disco persistente.

## Il build live, passo-passo



#### Credenziali raccolte

Shopify token, Triple Whale key, Service Account JSON, Drive folder IDs, Telegram bot, Gmail app password



#### Cartelle Drive condivise

con l'email del service account (lettura)



#### config.json compilato

chiavi API + matrix COGS + folder IDs



#### Agent buildato con la factory

/agent-spec → tasks → execute → verify



#### Test modalità report

data di ieri in TEST → Excel + Drive + Telegram



#### Modalità live testata

/api/metrics ok · login ok · la dashboard carica



#### Schedulato 16:00 + deploy live

cron del cloud + web service su Render, dominio + password

## Il take-away

Un solo agent, due modalità: la dashboard live nel browser e il report giornaliero su Drive e Telegram, dalle stesse fonti e dallo stesso `config.json`. Le credenziali prima, la spec dettata alla factory, le hard rules dentro la spec. Costruito una volta, lavora ogni giorno — anche quando non ci sei.